

B104: Artificial Intelligence and Machine Learning

Task: End to End Machine Learning Project

Title:

Automotive Client Purchase Prediction Agent

Dataset: [Kaggle Car Purchase Prediction](#)

Github: [Github Link](#)

Student Name: Charles Mwangi Ndungu

Student ID: GH1041919

Program: BSc Computer Science

Institution: Gisma University of Applied Sciences

Date: 18/09/2026

Artificial Intelligence and Machine Learning

End to end Automotive Purchase Prediction Machine learning Pipeline

1. Problem Statement

In today's world, the automotive retail industry is finding it hard to find, keep and attract new promising clients. Digital marketing campaigns often end up being a waste of time and money with actually low impact. My project focuses on solving this predicament by predicting the likelihood of a client to buy a car based on their demography, financial capacity and the product data.

By building this predictive model, a business is able to identify high priority buyers before sales outreach therefore saving and redirecting company resources to other key areas such as:

1. Marketing Budget Optimization: Shift digital marketing budget spent on low yield demographics towards high priority customer segments, lowering the customer acquisition cost (CAC).

2. Salesforce efficiency: Automatically prioritize lead queues in Customer Relationship Management (CRM) Platforms, allowing sales representatives to focus on top-tier prospects.

3. Better Inventory Distribution: Align regional vehicle allocation with local demographic demand profiles.

Machine Learning Problem Framing

We framed this business challenge as a standard supervised binary classification task. The target label, Purchased, is defined as:

$$y \in \{0, 1\}$$

where:

- $y = 1$: The customer bought a vehicle.
- $y = 0$: The customer did not buy a vehicle.

Our input features X consist of tabular data covering basic demographics (Age, Gender, Salary) and vehicle sales figures (Price, Sales).

Data Sourcing Strategy

Data comes directly from standard business data pipelines:

- Digital Touchpoints: Web form submissions, online vehicle configurators, and test-drive requests.
- CRM Records: Historical sales logs stored in internal enterprise databases.
- Demographic Enrichment: Anonymized third-party credit and income tier estimates.

2. Data Exploration & Quality Checks

Dataset Inspection

We ran initial Exploratory Data Analysis (EDA) across all features to verify data quality:

- **Missing Value Audits:** Checked for missing or null entries to see where imputation was necessary.
- **Schema Validation:** Verified data types across categorical variables (Gender) and numeric metrics (Age, Salary, Price, Sales).
- **Scale Discrepancies:** Numeric ranges varied wildly. Salary and Price stretch into the tens or hundreds of thousands, while Age stays strictly between 18 and 70. Distance- and gradient-based models (like KNN, Logistic Regression, and SVMs) need feature normalization so large numerical ranges don't overwhelm the loss function during training.

Class Imbalance & Resampling

Looking at Purchased, we found an uneven split between buyers ($y=1$) and non-buyers ($y=0$). Models trained on imbalanced data tend to favor the majority class, scoring high overall accuracy while completely missing the target buyers.

To fix this without throwing away useful majority-class records, we used the Synthetic Minority Over-sampling Technique (SMOTE). SMOTE generates synthetic data points along the line segments connecting k -nearest neighbors within the minority class:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda (\mathbf{x}_{zi} - \mathbf{x}_i), \quad \lambda \sim U(0, 1)$$

Choosing the Right Evaluation Metrics

Raw accuracy is misleading when classes are imbalanced because a dumb model that guesses "did not buy" every time can still look accurate. We evaluated models across a fuller set of performance metrics instead:

- Precision: Measures how often positive predictions are right:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Recall (Sensitivity): Measures the percentage of actual buyers caught by the model:

$$\text{Recall} = \frac{TP}{TP + FN}$$

From a business perspective, Recall is critical: missing a real buyer (False Negative) means lost sales revenue.

- F1-Score: Balances Precision and Recall into a single harmonic mean:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Confusion Matrix: Tracks raw counts for True Positives, False Positives, True Negatives, and False Negatives.

3. Data Preprocessing & Feature Engineering

Data Split Setup

To completely avoid data leakage, every preprocessing step (calculating means, standard deviations, and encoding maps) was fitted solely on the training set (X_{train}). The test set

(X_test) was kept completely isolated and transformed using parameters learned strictly from training.

Plaintext

Full Dataset

|

|——> Training Set (X_train, y_train) ——> Fit Preprocessor & SMOTE ——> Train

Models

|

|——> Test Set (X_test, y_test) —————> Transform Only

—————> Evaluate

Preprocessing Pipeline (ColumnTransformer)

We organized preprocessing steps using scikit-learn's ColumnTransformer:

- Categorical Features (Gender): Encoded using OneHotEncoder(drop='first', handle_unknown='ignore') to keep redundant columns out.
- Numeric Features (Age, Salary, Price, Sales): Transformed with PowerTransformer(method='yeo-johnson'). This stabilizes variance, reduces skewness, and pushes continuous distributions closer to a normal Gaussian curve.

Resampling the Training Set

SMOTE was applied only to the transformed training data (`X_train_transformed`, `y_train`). The test set (`X_test_transformed`) was left completely untouched to make sure our final evaluations reflect real-world distributions.

4. Model Benchmarking & Tuning

To find the best setup, we tested a range of linear, distance-based, kernelized, and tree-based models. Each model was tuned using `RandomizedSearchCV` with 5-fold cross-validation (`cv=5`).

Python

```
# Setup for RandomizedSearchCV cross-validation

from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(

    estimator=model_instance,

    param_distributions=param_grid,

    cv=5,

    random_state=42,

    n_jobs=-1,

)
```

search.fit(X_train_resampled, y_train_resampled)

Model Setup & Search Spaces

- K-Nearest Neighbors (KNN):
 - Method: Instance-based, non-parametric lookup.
 - Tuned Parameters: Neighbors $k \in [1, 20]$, distance weighting (uniform, distance).
- Support Vector Classifier (SVC):
 - Method: Finds optimal maximum-margin hyperplanes.
 - Tuned Parameters: Regularization $C \in [0.001, 100]$, $\gamma \in [0.001, 100]$, kernels (linear, rbf, poly).
- Logistic Regression:
 - Method: Linear decision boundary converted via log-odds.
 - Tuned Parameters: Regularization $C \in [0.001, 100]$, max iterations $\text{max_iter} \in [50, 700]$.
- Random Forest Classifier:
 - Method: Bagged ensemble of decision trees.
 - Tuned Parameters: Tree count $n_estimators \in [50, 300]$, depth $\text{max_depth} \in [4, 16]$, split metrics (gini, entropy), feature sampling (sqrt, log2).
- Gradient Boosting & HistGradientBoosting:
 - Method: Sequential additive boosting.
 - Tuned Parameters: Learning rate $\eta \in [0.05, 1.0]$, tree count, leaf constraints.

- Voting Classifier (Meta-Ensemble):
 - Method: Combines top base models (Random Forest, SVC, Gradient Boosting) into a hard-voting ensemble (voting='hard') to smooth out individual model errors.

5. Evaluation & Performance Comparison

All final models were benchmarked on the held-out test set (X_test_transformed).

Architecture	Precision (Class 1)	Recall (Class 1)	F1-Score (Macro)	Accuracy	Key Parameters
Logistic Regression	~0.82	~0.84	~0.83	~0.84	\$C=1.0\$, solver='lbfgs'
K-Nearest Neighbors	~0.86	~0.88	~0.87	~0.87	n_neighbors=7, weights='distance'
Support Vector Classifier	~0.93	~0.92	~0.93	~0.93	\$C=100\$, \$\gamma=1.0\$, kernel='rbf'
Random Forest	~0.94	~0.93	~0.94	~0.94	n_estimators=300, max_depth=8
Gradient Boosting	~0.93	~0.94	~0.94	~0.94	learning_rate=0.05, n_estimators=50
Voting	~0.95	~0.95	~0.95	~0.95	Ensemble (RFC +

Classifier	94	5	.95	SVC + GBC)
-------------------	----	---	-----	------------

Key Takeaways

- **Linear Baseline:** Logistic Regression established a decent baseline (~84% accuracy), but linear boundaries struggled to capture complex relationships, lowering recall.
- **Non-Linear Kernels:** Switching SVC to an RBF kernel bumped performance significantly (~93% F1-score).
- **Ensemble Top Performers:** Tree ensembles and the Voting Classifier performed best overall, successfully capturing non-linear interactions between salary levels, age brackets, and vehicle prices.

6. Final Insights & Recommendations

Strengths & Validation

- **Leak-Free Setup:** Keeping fitting routines (X_{train}) completely separate from test data (X_{test}) prevents overly optimistic performance estimates.
- **Effective Imbalance Management:** Using SMOTE balanced the class distribution without discarding training data.
- **High Precision:** The final Voting Classifier drastically cuts false positives while catching over 94% of true purchasing leads.